

FUNDAMENTALS OF DATA STRUCTURE
AND ALGORITHM

Project Report

Normal-2: A+B with Binary Search Trees

Date of Completion:

April 5, 2026

Chapter 1 Introduction

1.1 Background and Motivation

Binary Search Trees (BSTs) are among the most fundamental data structures in computer science. They provide efficient average-case $O(\log n)$ operations for search, insertion, and deletion, and their sorted structure enables elegant algorithms for problems involving ordered data. Understanding BST construction and traversal is therefore a core competency in any data-structures curriculum.

This project extends that understanding to a two-tree search scenario: given two independently constructed BSTs and a target integer N , we must find all pairs (A, B) such that A belongs to the first tree, B belongs to the second tree, and $A + B = N$. The problem combines BST construction from a non-standard parent-index encoding, multi-tree traversal, and efficient complementary-key look-up — making it an excellent exercise in algorithm design and data-structure integration.

1.2 Problem Description

We are given two BSTs, $T1$ and $T2$, and a target integer N . Each BST is described by a list of nodes, where each node is specified by its key value and the index of its parent node. The root node has a parent index of -1 .

A Binary Search Tree satisfies the following recursive definition:

- The left subtree of a node contains only nodes with keys strictly less than the node's key.
- The right subtree of a node contains only nodes with keys greater than or equal to the node's key.
- Both subtrees must themselves be valid BSTs.

The task is to:

1. Find all pairs (A, B) where A is a key in $T1$, B is a key in $T2$, and $A + B = N$.
2. If at least one solution exists, print true followed by all solutions in ascending order of A ; identical equations must appear only once.
3. If no solution exists, print false.
4. In all cases, print the preorder traversal sequences of $T1$ and $T2$.

1.3 Input / Output Specification

Input format:

- First line: $n1$ (number of nodes in $T1$, $1 \leq n1 \leq 2 \times 10^5$).

- Next $n1$ lines: key value and parent index for each node (keys in range $[-2 \times 10^9, 2 \times 10^9]$).
- T2 follows in the same format.
- Final line: target integer N (same range as keys).

Output format:

- "true" or "false" on the first line.
- If true: each solution on its own line, formatted as " $N = A + B$ ", in ascending order of A .
- Two preorder traversal lines: T1 then T2, values separated by single spaces, no leading/trailing space.

1.4 Constraints Summary

Parameter	Value
Max nodes per tree	2×10^5
Key range	$[-2 \times 10^9, 2 \times 10^9]$
Target N range	$[-2 \times 10^9, 2 \times 10^9]$
Max sum $A+B$ (long long required)	$\pm 4 \times 10^9$

Chapter 2 Algorithm Specification

2.1 Data Structures

2.1.1 BST Node Array (array-based representation)

Each tree is stored as a flat static array of Node structs, indexed by the node's input order number. We choose an array-based representation rather than a pointer-based one because the input already identifies nodes by integer index, allowing $O(1)$ random access without pointer indirection and avoiding per-node dynamic memory allocation.

Each Node contains:

- **key** — a 64-bit signed integer (long long). Individual keys fit in a 32-bit int, but the complement computation $N - A$ can reach $\pm 4 \times 10^9$, which overflows a 32-bit type.
- **left** — index of the left child (-1 if absent).
- **right** — index of the right child (-1 if absent).

```
typedef struct Node {
    long long key; /* key value */
    int left; /* index of left child (-1) */
    int right; /* index of right child (-1) */
} Node;

static Node BST1[MAXN]; /* both trees live in static storage */
static Node BST2[MAXN]; /* avoids stack overflow in main */
```

2.1.2 Global Result and Traversal Arrays

All large arrays that accumulate results across function calls are declared at file scope (global/static), not inside any function. This is a deliberate stack-safety measure: a local array of size $\text{MAXN} = 200001$ elements would consume up to 1.6 MB of stack space per call frame, risking stack overflow on systems with the common 8 MB default stack limit.

Global array	Size	Purpose
inorder1[]	$\text{MAXN} \times 8 \text{ B}$	Sorted T1 keys from inorder traversal
preorder1[], preorder2[]	$\text{MAXN} \times 8 \text{ B}$ each	Preorder sequences for output
result[]	$\text{MAXN} \times 8 \text{ B}$	Valid A values forming $A+B=N$ solutions

2.1.3 Hash Table Struct (hs_table)

T2's keys are stored in a dynamically heap-allocated open-addressing hash table, wrapped in a struct for clean ownership and parameter passing:

```
typedef struct hs_table {
    long long* head; /* heap-allocated slot array (HASH_SIZE entries) */
    int tlen; /* total capacity = 1 048 576 (2^20, 1 << 20, or 0x10000LL, anyway,
they are the same) */
    int cnt; /* number of distinct keys inserted */
} hs_table;
```

Design choices:

- Capacity 2^{20} : with $\leq 2 \times 10^5$ insertions, the load factor is ≈ 0.19 , which is acceptable in linear probing.
- Empty sentinel EMPTY = 0x80000001LL: this value exceeds the 32-bit signed integer range, so no valid key can ever collide with it.
- Hash function: XOR-fold the upper 32 bits of the key into the lower 32 bits before masking, for even distribution of large and negative keys.
- Collision resolution: linear probing with bit-AND modulo (HASH_MASK = HASH_SIZE - 1), which is faster than integer division.

Hash function pseudo-code:

```
hash_func(key):
    u = reinterpret key as unsigned 64-bit
    u = u ^ (u >> 32) // fold high bits into low bits
    return u & (HASH_SIZE - 1) // mask to [0, HASH_SIZE)
    // return value is always >= 0, no negative-check needed
```

2.1.4 Explicit Stack Struct

To prevent system-stack overflow on degenerate (fully skewed) trees of height up to $n = 2 \times 10^5$, all traversals are implemented iteratively with an explicit stack heap-allocated per traversal call:

```
typedef struct stack {
    int stk[MAXN]; /* node-index storage array */
    int top; /* index of top element; -1 = empty */
} stack;
```

All stack operations (push, pop, top, is_empty) run in $O(1)$. The struct is created with stack_create() (malloc) and freed with stack_destroy() (free) at the end of each traversal, preventing memory leaks.

2.2 Key Algorithms

2.2.1 Building a BST from Parent-Index Input (tree_create)

Because a parent node may appear after its child in the input stream, child pointers cannot be set during reading. Two sequential passes are required. Crucially, the temporary key and parent-index

arrays are heap-allocated inside the function (not stack-allocated) to avoid large local frames:

```
tree_create(tree[], n) -> root_index:
    keys[] = malloc(n * sizeof(long long)) // heap, not stack
    par[] = malloc(n * sizeof(int)) // heap, not stack
    // Pass 1: read all nodes
    for i in 0..n-1:
        scanf key[i], par[i]
        tree[i] = {key[i], left=-1, right=-1}
        if par[i] == -1: root = i
    // Pass 2: wire parent -> child links
    for i in 0..n-1:
        if par[i] == -1: continue
        p = par[i]
        if key[i] < tree[p].key: tree[p].left = i
        else: tree[p].right = i
    free(keys); free(par)
    return root
```

Time: $O(n)$. Space: $O(n)$ heap for the temporary arrays, freed before return.

2.2.2 Iterative Inorder Traversal (inorder)

Inorder traversal (left $\rightarrow$ root $\rightarrow$ right) of a BST yields keys in ascending order. The explicit stack simulates the recursion call frames:

```
inorder(tree[], root):
    stack = stack_create() // heap-allocated
    cur = root
    while cur != -1 OR stack is not empty:
        while cur != -1:
            push(stack, cur);
            cur = tree[cur].left // descend left
        cur = top(stack);
        pop(stack); // visit
        inorder1[size++] = tree[cur].key
        cur = tree[cur].right // move right
    stack_destroy(stack)
```

Time: $O(n1)$. Stack depth: $O(h1)$, worst case $O(n1)$ for a degenerate tree.

2.2.3 Iterative Preorder Traversal (preorder)

Root is visited first; right child is pushed before left so left is processed first (LIFO):

```
preorder(tree[], root, pre[], &size):
    stack = stack_create(); push(stack, root)
    while s is not empty:
```

```

    cur = top(stack);
    pop(stack);
    pre[size++] = tree[cur].key          // visit
    if right[cur] != -1: push(stack, right[cur])
    if left[cur] != -1: push(stack, left[cur])
    stack_destroy(stack)
    /* easy */

```

Time: $O(n)$. Stack depth: $O(h)$.

2.2.4 Mapping T2 Keys into the Hash Table (map_tree_to_hash)

An iterative traversal visits every node of T2 and inserts its key. The traversal order is irrelevant for hash correctness. In-order is used here for consistency. The key correctness requirement is that after visiting each node the cursor advances to the right subtree:

```

map_tree_to_hash(tree[], root, table):
    stack = stack_create(); cur = root
    while cur != -1 OR stack is not empty:
        while cur != -1:
            push(stack, cur);
            cur = tree[cur].left
        cur = top(stack);
        pop(stack)
        hash_insert(table, tree[cur].key)
        cur = tree[cur].right
    stack_destroy(stack)

```

Time: $O(n^2)$ expected. Space: $O(h^2)$ for the stack.

2.2.5 Finding All Pairs $A + B = N$

With `inorder1[]` sorted and the hash table loaded, pairs are found in a single linear scan.

Consecutive equal A values are skipped via a one-element look-back comparison, ensuring each unique equation is emitted exactly once. No separate sort is needed because `inorder1[]` is already in ascending order:

```

find_pairs(inorder1[], N, table) -> result[]:
    for i in 0..inorder1_size-1:
        if i > 0 AND inorder1[i] == inorder1[i-1]: continue // skip dup
        a = inorder1[i]
        if hash_search(table, N - a): result[res_size++] = a
    // result[] is already sorted ascending

```

Time: $O(n^1)$ expected.

2.3 Overall Program Sketch

```
main():
    scanf n1; root1 = tree_create(BST1, n1)    // keys/par heap-allocated inside
    scanf n2; root2 = tree_create(BST2, n2)
    scanf N
    inorder(BST1, root1)                       // -> inorder1[] (sorted, global)
    table = hash_create(HASH_SIZE)             // 8 MB heap
    map_tree_to_hash(BST2, root2, table)
    find_pairs(inorder1[], N, table)           // -> result[] (global)
    hash_destroy(table)                       // free 8 MB
    print true/false and solutions from result[]
    preorder(BST1, root1, preorder1, &size1)  // -> preorder1[] (global)
    preorder(BST2, root2, preorder2, &size2)  // -> preorder2[] (global)
    print preorder1[], preorder2[]
```

All large arrays live in global/static storage (BSS segment). Heap allocations (hash table, stacks, temp arrays) are freed before program exit. No allocation of MAXN-sized arrays occurs on any function's stack frame.

Chapter 3 Testing Results

The program was compiled with GCC (gcc -O2 -Wall) and tested against seven distinct cases covering normal operation, edge cases, and boundary conditions. All seven cases passed.

TC	Purpose	Input Summary	Expected Output
1	Sample 1 — normal, multiple solutions	T1: 8 nodes; T2: 7 nodes; N=36	true + 3 solutions + preorders
2	Sample 2 — no solution	T1: 5 nodes; T2: 3 nodes; N=40	false + preorders
3	Minimum size: single-node trees	T1={5}, T2={5}, N=10	true, 10 = 5 + 5
4	Negative keys	T1={-5,-10,-3}, T2={-8}, N=-13	true, -13 = -5 + -8
5	No solution, small trees	T1={1,2}, T2={3,4}, N=100	false + preorders
6	Boundary: max+min keys, sum=0	T1={2e9}, T2={-2e9}, N=0	true, 0 = 2000000000 + -2000000000
7	Duplicate A values — dedup check	T1={5,5,5,10}, T2={15,20,5}, N=20	true, 20 = 5+15 (printed once only)

3.1 Test Case Details and Actual Output

Test Case 1 — Sample Input 1

Input: T1 has 8 nodes (root key=15); T2 has 7 nodes (root key=20); N = 36.

```
true
36 = 15 + 21
36 = 16 + 20
36 = 18 + 18
15 13 12 14 17 16 18 18
20 16 13 18 25 21 28
```

Result: PASS — three valid pairs found, output in ascending order of A.

Test Case 2 — Sample Input 2 (no solution)

Input: T1 = {10, 5, 15, 2, 7}, T2 = {15, 10, 20}, N = 40.

```
false
10 5 2 7 15
15 10 20
```

Result: PASS.

Test Case 3 — Minimum size: single-node trees

Purpose: verify no off-by-one errors at minimum input size ($n_1 = n_2 = 1$).

Input: $T_1 = \{5\}$, $T_2 = \{5\}$, $N = 10$.

```
true
10 = 5 + 5
5
5
```

Result: PASS.

Test Case 4 — Negative keys

Purpose: verify that long long arithmetic handles negative keys and negative sums correctly.

Input: $T_1 = \{-5, -10, -3\}$, $T_2 = \{-8\}$, $N = -13$.

```
true
-13 = -5 + -8
-5 -10 -3
-8
```

Result: PASS.

Test Case 5 — No solution, small trees

Input: $T_1 = \{1, 2\}$, $T_2 = \{3, 4\}$, $N = 100$.

```
false
1 2
3 4
```

Result: PASS.

Test Case 6 — Boundary maximum and minimum keys

Purpose: confirm that computing $N - A$ with $A = 2 \times 10^9$ and $N = 0$ does not overflow. A 32-bit int would overflow here; long long is required.

Input: $T_1 = \{2,000,000,000\}$, $T_2 = \{-2,000,000,000\}$, $N = 0$.

```
true
0 = 2000000000 + -2000000000
2000000000
-2000000000
```

Result: PASS

Test Case 7 — Duplicate A values (deduplication)

Purpose: confirm that when the same key appears multiple times in T1, the corresponding equation is printed exactly once. T1 contains three nodes with key=5 and one with key=10.

Input: T1 = {5(root), 5(left), 5(left-left), 10(right)}, T2 = {15, 5, 20}, N = 20.

```
true
20 = 5 + 15
5 10
15 5 20
```

Result: PASS — the equation '20 = 5 + 15' is emitted once despite three copies of A=5 in inorder1[].

Deduplication relies on the sorted property: the check 'inorder1[i] == inorder1[i-1]' correctly identifies and skips all consecutive duplicates.

Chapter 4 Analysis and Comments

4.1 Time Complexity Analysis

4.1.1 Building Each BST — $O(n)$

Two sequential passes over n nodes: reading with `scanf` ($O(n)$) and wiring child links ($O(n)$). Heap allocation and free of `keys[]` and `par[]` are $O(n)$. Both trees together: $O(n_1 + n_2)$.

4.1.2 Inorder Traversal of T_1 — $O(n_1)$

Each node is pushed and popped exactly once from the explicit stack. Total: $O(n_1)$. The resulting `inorder1[]` is non-decreasingly sorted by the BST property, enabling both ascending output order and $O(1)$ duplicate detection in the pair-finding loop.

4.1.3 Hash Table Construction from T_2 — $O(n_2)$ expected

Inserting n_2 keys into 2^{20} slots at load factor ≈ 0.19 gives an expected probe length of $1/(1-0.19) \approx 1.23$ per operation, so $O(1)$ expected per insert and $O(n_2)$ total.

4.1.4 Pair-Finding Loop — $O(n_1)$ expected

One pass through `inorder1[]` ($O(n_1)$ iterations). Each iteration does one $O(1)$ `hash_search` and one $O(1)$ equality comparison for duplicate skipping. Total: $O(n_1)$ expected.

4.1.5 Preorder Traversals and Output — $O(n_1 + n_2)$

Each node visited exactly once per traversal. Output loops are also $O(n)$. Total: $O(n_1 + n_2)$.

4.1.6 Overall Time Complexity

$T(n_1, n_2) = O(n_1 + n_2)$ (expected)

This is asymptotically optimal: every node of both trees must be read at least once, which already costs $O(n_1 + n_2)$.

4.2 Space Complexity Analysis

4.2.1 Static BST Arrays — $O(n_1 + n_2)$ active

`BST1[MAXN]` and `BST2[MAXN]` reside in the BSS segment (static storage). They are always allocated at program load, but only the first n_1 and n_2 entries are meaningfully used.

4.2.2 Global Result and Traversal Arrays — $O(n_1 + n_2)$ active

`inorder1[]`, `preorder1[]`, `preorder2[]`, and `result[]` are all static global arrays of size `MAXN`. Placing them at global scope instead of inside function frames is a deliberate design choice to prevent stack overflow: a single local array of `MAXN` long longs consumes 1.6 MB of stack, and with

multiple such functions active simultaneously the total stack usage could easily exceed the typical 8 MB system limit.

4.2.3 Heap-allocated Hash Table — $O(\text{HASH_SIZE})$

`hash_create` allocates $\text{HASH_SIZE} \times \text{sizeof}(\text{long long}) = 2^{20} \times 8 = 8 \text{ MB}$ on the heap. It is freed by `hash_destroy` after the pair-finding loop completes.

4.2.4 Heap-allocated Traversal Stacks — $O(h)$ active depth

Each stack struct holds `MAXN` ints ($\approx 800 \text{ KB}$), heap-allocated and freed per traversal call. At most one stack is live at any given time. The worst-case depth is $O(n)$ for a degenerate tree.

4.2.5 Heap-allocated Temporary Arrays in `tree_create` — $O(n)$ transient

`keys[]` and `par[]` are `malloc`'d inside `tree_create` and freed before the function returns. They are transient: at peak they consume $O(n) \times (\text{sizeof}(\text{long long}) + \text{sizeof}(\text{int})) \approx 2.4 \text{ MB}$ per call. Allocating these on the heap rather than the stack is essential: stack-local arrays of size `MAXN` would add another 2.4 MB per `tree_create` call frame, totalling $\approx 5 \text{ MB}$ of extra stack usage across two calls — a likely overflow on systems with 8 MB stacks.

4.2.6 Overall Space Complexity

$S(n_1, n_2) = O(n_1 + n_2 + \text{HASH_SIZE}) = O(n_1 + n_2)$ asymptotically.

In absolute terms: $\approx 5 \times \text{MAXN} \times 8 \text{ B}$ static arrays + 8 MB hash table + 800 KB stack $\approx 22 \text{ MB}$ total, all within typical online-judge memory limits of 256 MB.

4.3 Stack Overflow: a Hidden Pitfall

A particularly subtle class of bugs in this problem is stack overflow caused by large local arrays. The table below summarises the three locations where stack-unsafe arrays were identified during development, and how each was resolved:

Location	Unsafe declaration	Stack cost	Fix applied
<code>main()</code>	<code>long long result[MAXN]</code>	1.6 MB	Moved to global scope
<code>tree_create()</code> $\times 2$ calls	<code>long long keys[MAXN]</code>	$1.6 \text{ MB} \times 2 = 3.2 \text{ MB}$	Replaced with <code>malloc</code> / <code>free</code>
<code>tree_create()</code> $\times 2$ calls	<code>int par[MAXN]</code>	$0.8 \text{ MB} \times 2 = 1.6 \text{ MB}$	Replaced with <code>malloc</code> / <code>free</code>

. The fixes are invisible in algorithmic complexity but critical in practice.

4.4 Comparison with Alternative Approaches

Algorithm A — Sort T2 + Binary Search

- Time: $O((n_1 + n_2) \log n)$ — extra log factor per look-up.
- Space: $O(n_1 + n_2)$.
- Advantage: deterministic; no hash collisions possible.

Algorithm B — Two-pointer on Both Sorted Arrays

- Time: $O(n_1 + n_2)$ — same asymptotic as our approach.
- Space: $O(n_1 + n_2)$ — no hash table needed.
- Advantage: deterministic; elegant when no duplicates exist.
- Disadvantage: extra deduplication logic needed for repeated keys; more complex loop invariants.

Our chosen approach (hash table + sorted inorder sweep) achieves $O(n_1 + n_2)$ expected time. The struct-based design (hs_table, stack) with explicit create/destroy pairs makes ownership clear and memory leaks easy to solve~ :-)

4.5 Implementation Notes and Lessons

- long long everywhere: individual keys fit in int, but $N - A$ can reach $\pm 4 \times 10^9$, requiring 64-bit arithmetic throughout.
- Iterative traversal: degenerate BSTs can have height $n = 2 \times 10^5$. Recursive traversal would overflow the system stack; an explicit heap-allocated stack avoids this.
- EMPTY = 0x80000001LL: this sentinel lies outside the 32-bit signed integer range guaranteed for all keys. No valid key can equal it.
- Global large arrays: any array of MAXN elements must live in global/static storage, not on the stack. This lesson emerged from debugging stack overflow during development.
- Heap temp arrays in tree_create: even temporary arrays used only inside a function must be heap-allocated if they exceed a few hundred kilobytes, because each call frame's local variables accumulate on the stack.
- Struct encapsulation: wrapping hash table and stack in structs with matching create/destroy makes resource ownership explicit and simplifies memory-leak detection.

Appendix Key Code Excerpts

The complete source file `bst.c` is included in the submission ZIP. Key excerpts are reproduced here.

Global arrays (stack-safety)

```
/* All MAXN-sized arrays at global scope – never on the stack */
static long long inorder1[MAXN];
static long long preorder1[MAXN], preorder2[MAXN];
static long long result[MAXN]; /* moved from main() to avoid sys-stack overflow */
```

tree_create

```
static int tree_create(Node tree[], int n) {
    long long* keys = malloc(sizeof(long long) * n); /* heap, not stack */
    int* par = malloc(sizeof(int) * n);
    /* ... read, wire, then: */
    free(keys); free(par);
    return root;
}
```

hash_insert

```
static void hash_insert(hs_table* table, long long key) {
    int idx = hash_func(key), org_idx = idx;
    while (table->head[idx] != EMPTY && table->head[idx] != key) {
        idx = (idx + 1) & HASH_MASK;
        If(idx == org_idx) {
            Return;
        }
    }
    if (table->head[idx] == EMPTY) { /* truly new key */
        table->head[idx] = key;
        table->cnt++;
    }
}
```

map_tree_to_hash

```
cur = stack_top(stack); stack_pop(stack);
hash_insert(table, tree[cur].key);
cur = tree[cur].right;
```

Pair-finding loop

```
for (int i = 0; i < inorder1_size; i++) {
    if (i > 0 && inorder1[i] == inorder1[i-1]) continue; /* skip dup A */
    long long a = inorder1[i];
```

```
    if (hash_search(table, N - a))  
        result[res_size++] = a;  
}
```